PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Construcción y documentación de una API con Express

Presentado a: Instructora Alexandra Muñoz

Por Aprendiz: **Nicolas Ruiz Colorado**

Ficha: 3312932

Competencia:

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **20** de **octubre** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

Actividad Construcción y documentación de una API con express 6

1. Creación del proyecto 6

1.1. Creación de la carpeta del proyecto 6

1.2. Creación del proyecto con npm init -y 6

1.3. Instalación del npm express 7

1.4. Instalar el nodemon 7

2. Configuración de ejecución del servidor en modo normal y en modo desarrollador con Nodemon. 8

2.1. Ejecución en modo normal 8

2.2. Ejecución en modo nodemon 8

3. Crear una ruta get/inicio que devuelva un mensaje de bienvenida en formato JSON 8

4. Crear una ruta get/información que devuelva información general del servicio en formato JSON 9

5. Crear una ruta post/solicitudes que reciba nombre, apellido y correo electrónico en el cuerpo de la petición 9

6. Validar que los tres campos sean enviados y que el correo tenga un formato razonable. Si los datos no cumplen la API debe responder de forma clara 9

7. Cuando los datos sean válidos, responder que la solicitud fue recibida. No es obligatorio guardar datos en una base de datos. 10

8. Incorporar al menos un middleware propio que registre en la consola el método HTTP y la ruta solicitada 10

9. Aplicar el middleware de análisis de JSON que ofrece Express, y justificar por qué debe registrarse antes de las rutas que leen req.body 10

10. Configurar respuestas para rutas inexistentes y para solicitudes con datos inválidos 11

11. Investigación obligatoria 11

11.1. ¿Qué es un middleware en Express y en que momento del ciclo de una petición se ejecuta?..... 11

11.2. ¿Qué significan los objetos req y res? 11

11.3. ¿Cuál es la diferencia entre un middleware global, uno aplicado a una ruta especifica y un middleware de manejo de errores?..... 12

11.4. ¿Qué hace express.json() y qué problema se presenta si no se configura cuando el cliente envía el JSON? 13

11.5. ¿Por qué importa el orden en que se registran los middlewares y las rutas? 13

11.6. ¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo? 13

11.7. ¿Cuál es la diferencia entre un middleware y la función que atiende una ruta? 14

11.8. ¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite? 14

11.9. Incluye un ejemplo cotidiano, explicado sin código que ayude a entender la función de un middleware. 14

12. Pruebas de la API 15

12.1. GET/inicio: verificar que devuelva una respuesta JSON 15

12.2. GET/información: verificar que devuelva la información general 16

12.3. POST/solicitudes con nombre, apellido y correo validos 17

12.4. POST/solicitudes sin uno o más campos obligatorios 18

12.5. POST/solicitudes con correo de formato invalido 19

12.6. Consultar una ruta que no exista y verificar la respuesta 404 20

12.7. Verificar en la consola que el middleware propio registre las peticiones 21

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL

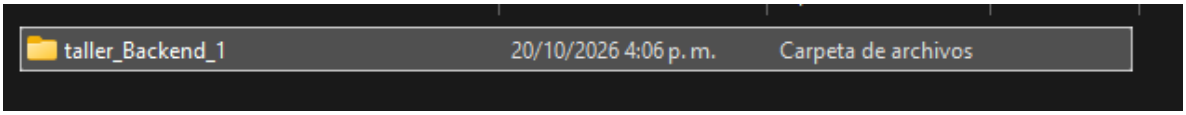
FORMATO ENTREGA DE EVIDENCIAS

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Construcción y documentación de una API con express

1. Creación del proyecto

1.1. Creación de la carpeta del proyecto



1.2. Creación del proyecto con npm init -y

```
Microsoft Windows [Versión 10.0.26200.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

E:\ADSO_SENA\5TO TRIMESTRE\backend\proyectos\taller_Backend_1>npm init -y
Wrote to E:\ADSO_SENA\5TO TRIMESTRE\backend\proyectos\taller_Backend_1\package.json:

{
  "name": "taller_backend_1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs"
}



E:\ADSO_SENA\5TO TRIMESTRE\backend\proyectos\taller_Backend_1>
```

1.3. Instalación del npm express

```
E:\ADSO_SENA\5TO TRIMESTRE\backend\proyectos\taller_Backend_1>npm install express

added 65 packages, and audited 66 packages in 32s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

E:\ADSO_SENA\5TO TRIMESTRE\backend\proyectos\taller_Backend_1>
```

1.4. Instalar el nodemon

```
E:\ADSO_SENA\5TO TRIMESTRE\backend\proyectos\taller_Backend_1>npm install --save-dev nodemon

added 26 packages, and audited 92 packages in 17s

33 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.

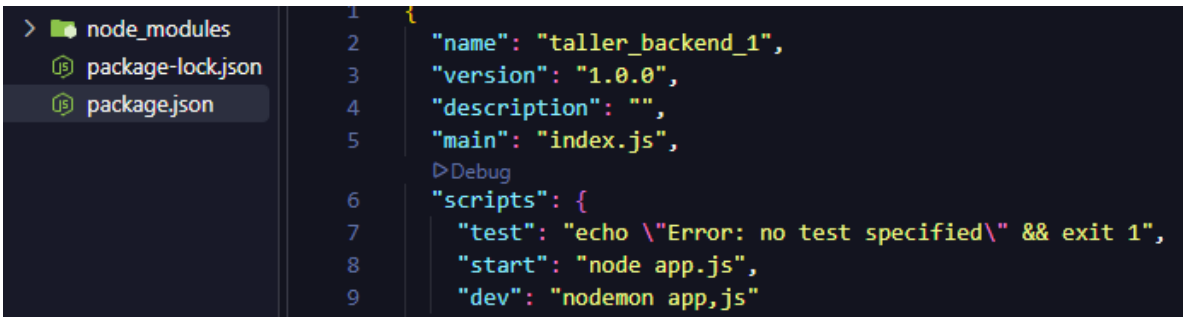
E:\ADSO_SENA\5TO TRIMESTRE\backend\proyectos\taller_Backend_1>
```

2. **Configuración de ejecución del servidor en modo normal y en modo desarrollador con Nodemon.**

2.1. Ejecución en modo normal



2.2. Ejecución en modo nodemon



3. **Crear una ruta get/inicio que devuelva un mensaje de bienvenida en formato JSON**

```
app.get('/inicio',(req,res)=>{
    res.json({mensaje:"Bienvenido a mi app"})
})
```

4. **Crear una ruta get/información que devuelva información general del servicio en formato JSON**

```
app.get('/informacion',(req,res)=>{
    res.json({
        nombre:"App prueba",
        version:"1.0.0",
        descripcion:"En esta app podremos encontrar informacion general y
recibir solicitudes"
    })
})
```

5. **Crear una ruta post/solicitudes que reciba nombre, apellido y correo electrónico en el cuerpo de la petición**

```
app.post('/solicitudes',(req,res)=>{
    const{nombre,apellido,correo}= req.body
})
```

6. **Validar que los tres campos sean enviados y que el correo tenga un formato razonable. Si los datos no cumplen la API debe responder de forma clara**

```
app.post('/solicitudes',(req,res)=>{
    const{nombre,apellido,correo}= req.body
    if(!nombre.trim() || !apellido.trim() || !correo.trim()){
        return res.status(400).json({
            error:"Debe diligenciar todos los campos"
        })
    }
    const regexCorreo = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
    if (!regexCorreo.test(correo)){
        return res.status(400).json({
            error: "El correo no tiene un formato válido"
        })
```

```
    }
    res.status(201).json({mensaje:"Solicitud recibida"})
})
```

7. **Cuando los datos sean válidos, responder que la solicitud fue recibida. No es obligatorio guardar datos en una base de datos.**

```
res.status(201).json({mensaje:"Solicitud recibida"})
```

8. **Incorporar al menos un middleware propio que registre en la consola el método HTTP y la ruta solicitada**

```
app.use((req,res,next)=>{
    console.log(req.method, req.url)
    next()
})
```

9. **Aplicar el middleware de análisis de JSON que ofrece Express, y justificar por qué debe registrarse antes de las rutas que leen req.body**

```
app.use(express.json())
```

Cuando un cliente envía un JSON, lo que llega por la red es texto crudo este middleware lo lee y lo convierte en un objeto de JavScript y lo guarda en req.body.

Este middleware debe ir primero porque Express ejecuta todo en el orden en que se escribe y cada petición recorre la lista de arriba hacia abajo y se detiene en la primera ruta que responde.

10. Configurar respuestas para rutas inexistentes y para solicitudes con datos inválidos

```
app.use((req,res)=>{
    res.status(404).json({
        error:"Ruta no encontrada"
    })
})
```

11. Investigación obligatoria

11.1. ¿Qué es un middleware en Express y en que momento del ciclo de una petición se ejecuta?

Es una función que se ejecuta durante el proceso de atención de una petición HTTP en express.

Este se ejecuta después de que el servidor recibe una petición

- https://expressjs.com/en/guide/using-middleware/

11.2. ¿Qué significan los objetos req y res?

Estos objetos se utilizan para la conexión entre cliente y servidor

- req: Abreviatura o variable que representa el request es la que envia la petición del cliente al servidor, tiene los datos o parámetros, la cabecera y el tipo de petición
- res: Abreviatura o valorable que representa la respuesta del servidor al cliente, permite devolver mensajes de información y los códigos de estado.

https://expressjs.com/en/5x/api/request/

https://expressjs.com/en/5x/api/response/

11.3. ¿Cuál es la diferencia entre un middleware global, uno aplicado a una ruta especifica y un middleware de manejo de errores?

- Global: permite aplicar tareas generales en diferentes rutas
- Ruta especifica: permite controlar procesos particulares
- Manejo de errores: centraliza el tratamiento de situaciones inesperadas

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction?utm_source=chatgpt.com#:~:text=There%20is%20a%20special,the%20http%20module).

11.4. ¿Qué hace express.json() y qué problema se presenta si no se configura cuando el cliente envía el JSON?

Permite recibir y convertir los datos que llegan al servidor en formato JSON para poder ser utilizados dentro de la aplicación.

Si este no se configura, el servidor no podrá interpretar automáticamente el contenido de la petición mediante el middleware.

https://expressjs.com/en/5x/api/express/#expressjson:~:text=The%20Express%20object,express.json()

11.5. ¿Por qué importa el orden en que se registran los middlewares y las rutas?

Porque su forma de ejecución es una ejecución secuencial, y algunos middlewares podrían necesitar de otros como por ejemplo un middleware que muestre algún mensaje informativo.

https://expressjs.com/en/guide/using-middleware/

11.6. ¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo?

La petición queda detenida es decir que el servidor no podría procesar la solicitud y que el cliente se quede esperando por una respuesta

https://expressjs.com/es/resources/middleware/timeout/

11.7. ¿Cuál es la diferencia entre un middleware y la función que atiende una ruta?

Un middleware es un intermediario entre el cliente y el servidor, el cual procesa, filtra o modifica una solicitud en cambio una función de ruta es el destino final encargado de generar y enviar la respuesta al cliente.

https://medium.com/@patilchetan2110/getting-started-with-basics-of-routes-and-middleware-in-express-js-90db1b8c2252

11.8. ¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite?

Podría haber accesos no autorizados, datos incorrectos, suplantación de identidad y errores en la aplicación, las validaciones y los controles de acceso son importantes para proteger la información y el funcionamiento de la aplicación.

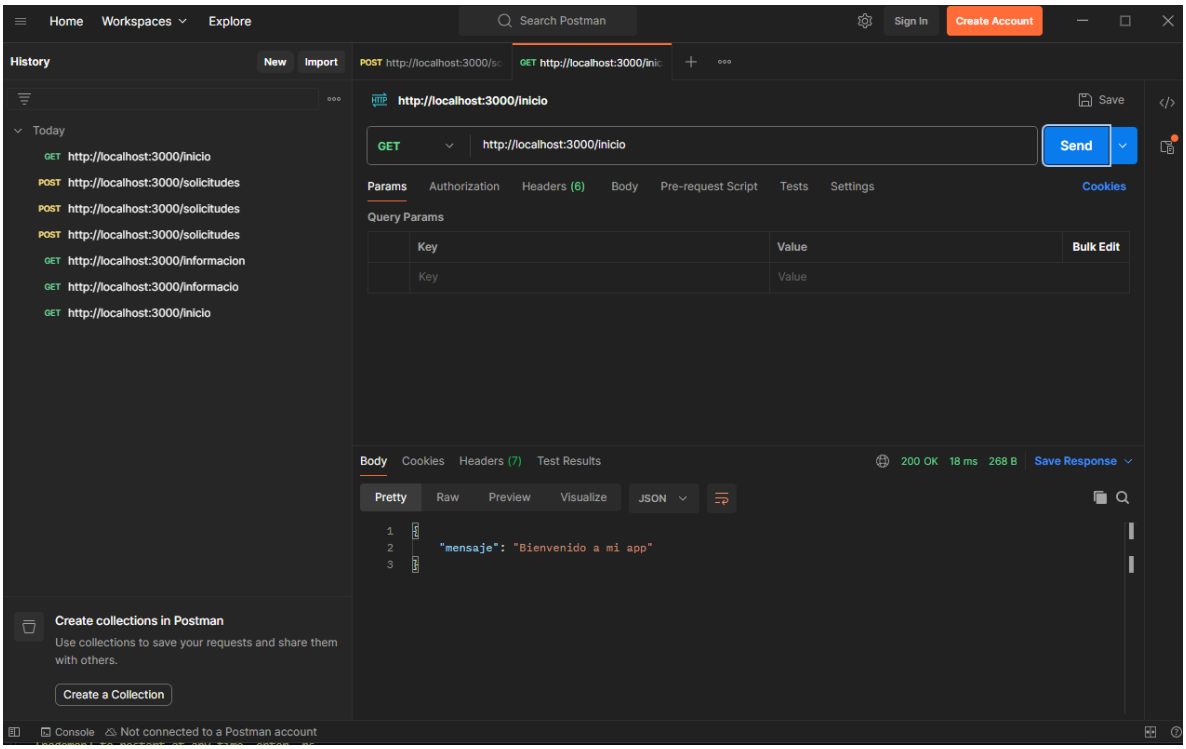
11.9. Incluye un ejemplo cotidiano, explicado sin código que ayude a entender la función de un middleware.

Para mi un ejemplo de un middleware podría ser un acceso de frontera, en este lugar tu tienes que presentar una documentación que te acredita el acceso; en este lugar comprueban tu identidad y si algo no está bien te puede denegar el acceso.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

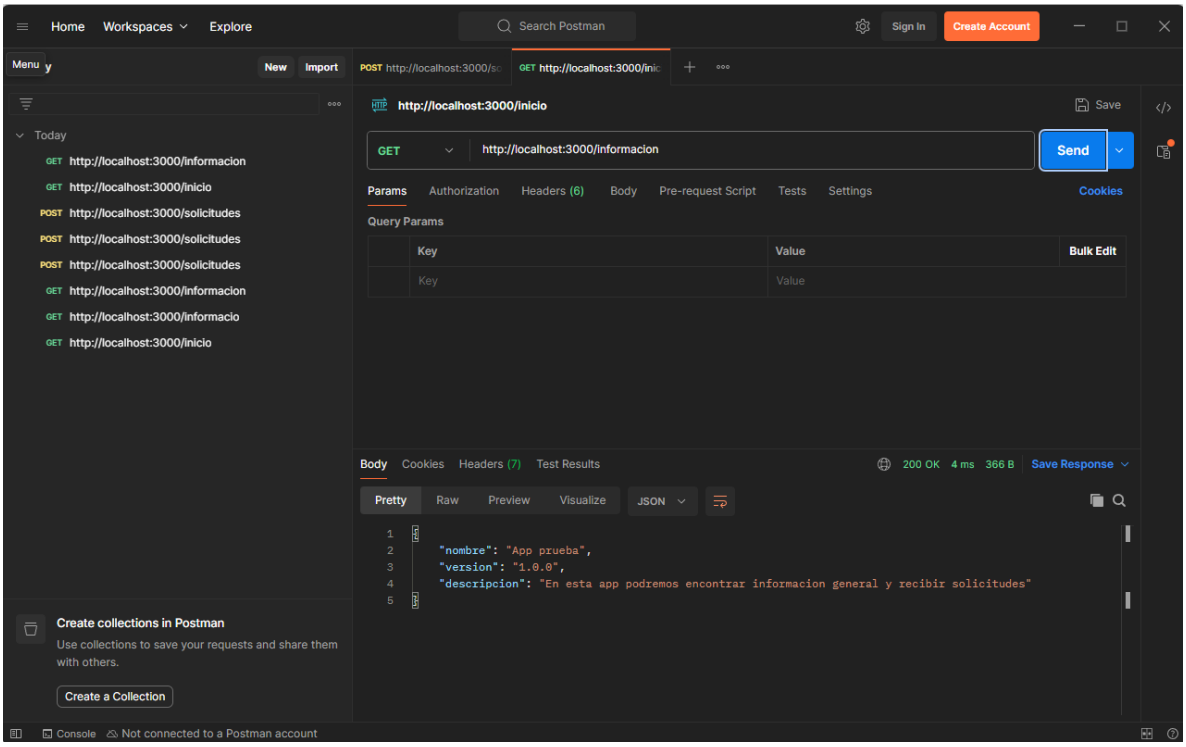
12. Pruebas de la API

12.1. GET/inicio: verificar que devuelva una respuesta JSON



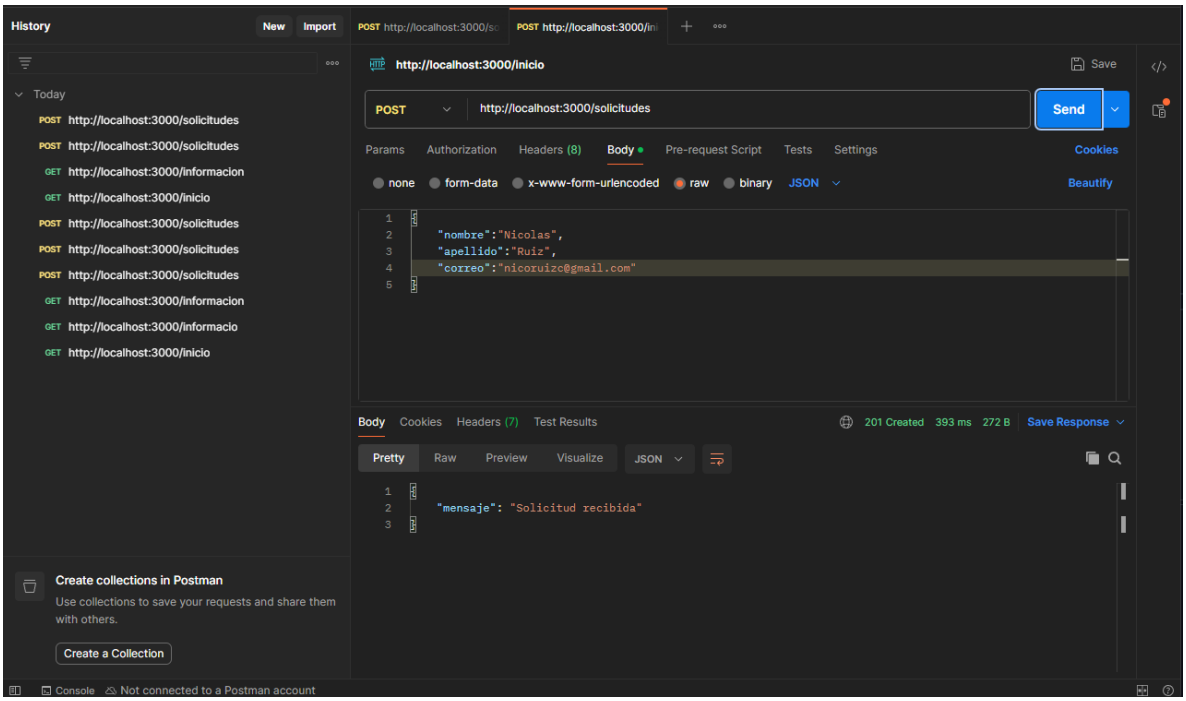
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

12.2. GET/información: verificar que devuelva la información general



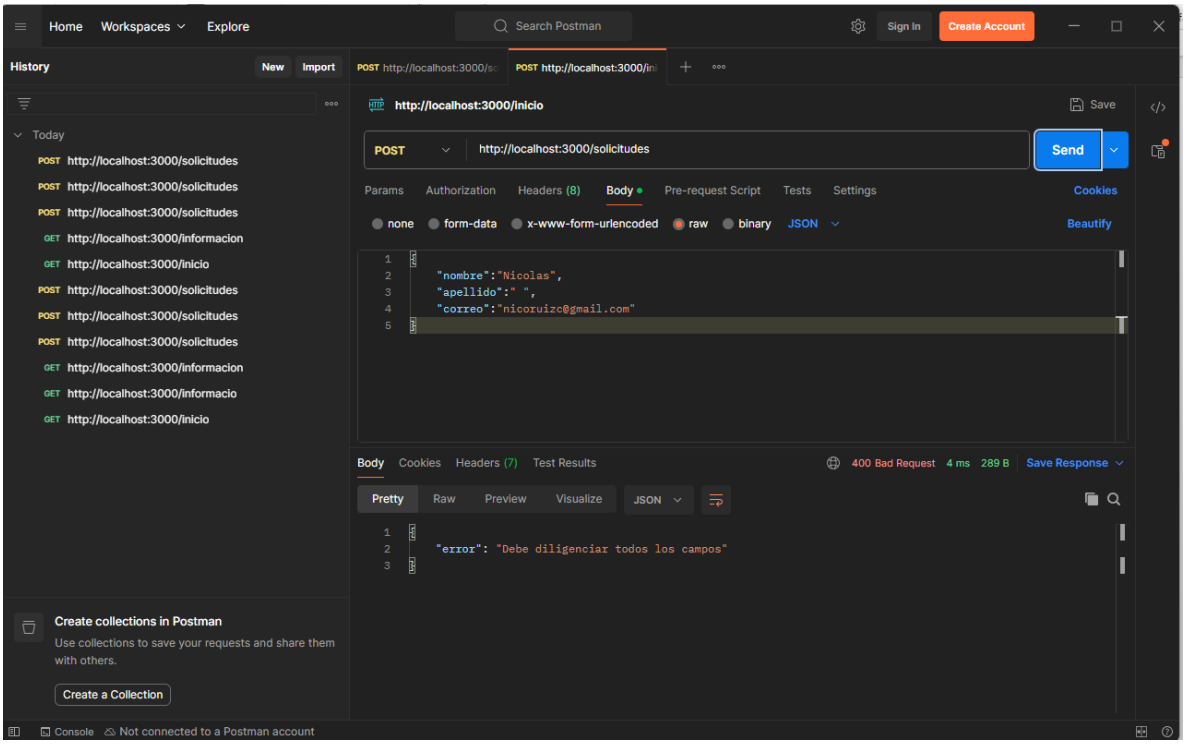
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

12.3. POST/solicitudes con nombre, apellido y correo validos



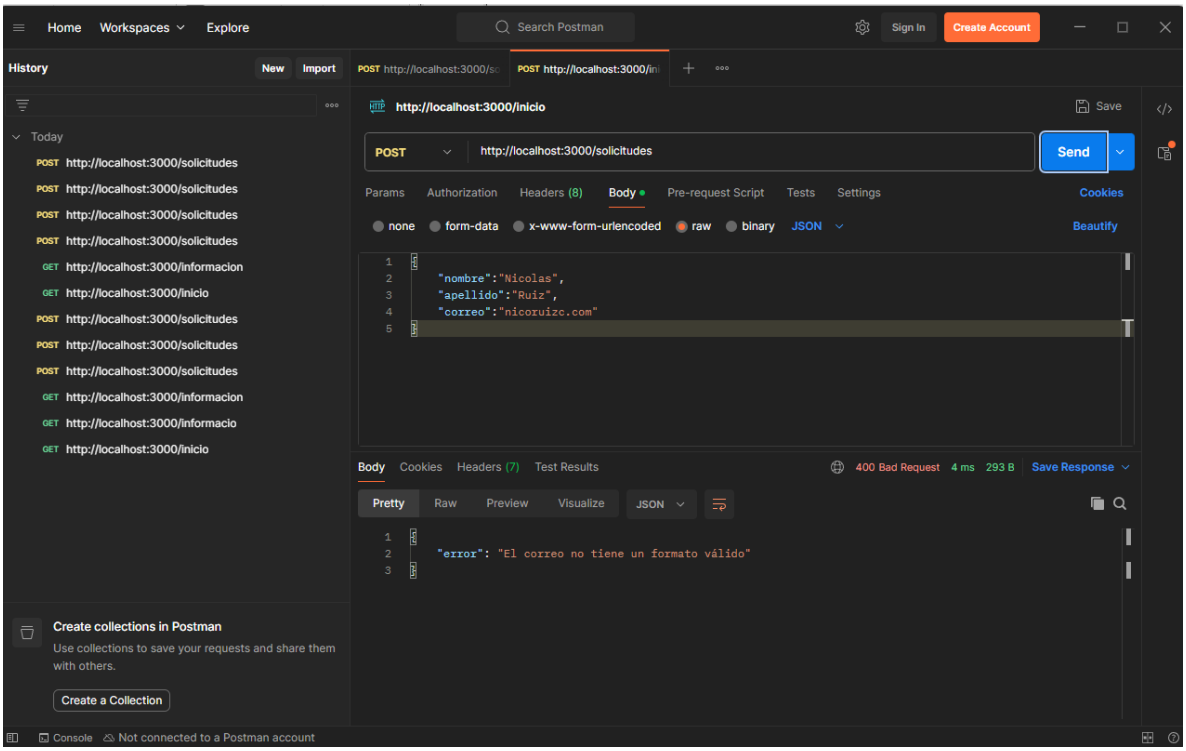
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

12.4. POST/solicitudes sin uno o más campos obligatorios



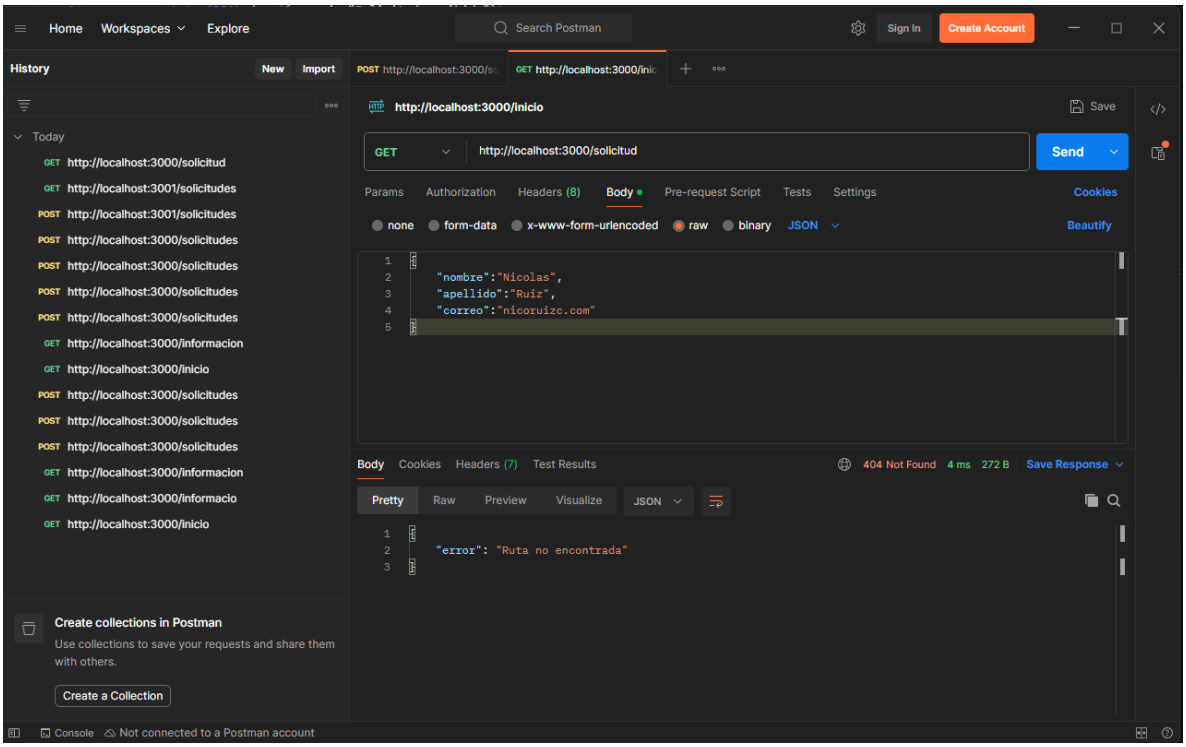
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

12.5. POST/solicitudes con correo de formato invalido



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

12.6. Consultar una ruta que no exista y verificar la respuesta 404



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

12.7. Verificar en la consola que el middleware propio registre las peticiones

